

Time Expectations:

We estimate it will take between four and six hours to complete this assignment. Whenever you're done, please document what your next steps would be. This isn't intended to be perfect, just something we can review and talk through with you.

Primary Task:

Develop an order ingestion and management system which can receive orders from multiple parallel sources and manage dispatch of orders to a robotic system for assembly. Included are a set of notional test data exemplifying different possible order sources and scenarios.

Product Requirements:

- Create a minimal frontend / backend system that merges and tracks food orders.
- Support both real time and scheduled orders.
- Provide views to show orders by status and any other relevant metadata.
- Provide a view of each order and a history of any changes or events that are relevant to that order.
- Skeleton a payload for orders that are dispatched to the robot.

Architectural Requirements:

- Support ingesting order data from three separate pipelines
 - Webhook-like (webhook.jsonl)
 - "Live" orders will be sent to your system, consider it to be a real time stream of user requests.
 - Assume a bursty traffic profile.
 - Each line of the sample data represents one order request payload.
 - Polling API Call (api.jsonl)
 - Calling an external API to request recent orders using a time_since query parameter.
 - The API will return deltas since last time_since, allow for scheduled orders. The API may return new orders, updates, and cancellations since the supplied time_since value.
 - Each line of sample data represents one api response received from this style of API.
 - CSV (orders_1.csv / orders_2.csv / orders_3.csv / orders_4.csv)
 - CSVs of orders can be expected to be uploaded multiple times per day.
 - Each line of sample data represents a single order provided by a user styled after the output of a survey form.
- Provide mocks for each of the above pipelines using the provided sample data.
- Provide a strategy for reasonable fault tolerance across the above pipelines.
- Provide a set of APIs for fetching historical order information.
- For simplicity when an order needs to be constructed by our robot, track it in a `dispatched` state.
- The system must be able to handle 100,000 requests per day

Deliverables:

- Design/architecture document in your choice (markdown / README is perfectly fine)
 - Considerations or any interesting decisions that went into your design choices (for example, maintainability, cost, language, scalability etc).
- A working skeleton of the system (frontend and backend)
 - There should be clear documentation for how to get it up and running.
 - Ideally the system should turn over – there should be something we could run
 - Ideally there should be an easy way to inject additional orders into the system using mocks for any of the webhook / api / csv style data sources.
- Sending a github link (or repo/source) and any screenshots
- When using AI, please include a log of the prompts / sessions that were used.